MOMENTUM

What to know before showing it to a professor, for a designer who does not write code

1. What you are, and what you are not

You are the designer and director of this game. The code was written by an AI build crew (Claude Code) to your specification, your rules and your review. You do not write C#, and you should not pretend to. That is a legitimate way to make software now, and it is the only honest account of this project. Nobody gets called a larper for using a tool. People get called larpers for claiming skills they do not have, or for not understanding their own project.

So the standard you have to meet is this: explain what the game is, why it is built the way it is, what the trade-offs were, what is broken, and what comes next. Those are a designer's questions and you can own every one of them.

> **Say this first, before anyone asks:**
>
> "I designed and directed this; the code is generated by Claude Code to my specification. I write the design documents and the rules the agents build against, I review every report, I play-test, and I send back what is wrong. I do not write C#. I can tell you what every system does and why it exists, and I can show you the process."

2. The one-breath pitch

MOMENTUM is a first-person movement shooter in the line of Bloodthief, ULTRAKILL and Mirror's Edge. Speed is the score, and the only way to earn more speed is harder timing. It is built in Unity 6 and everything in it is generated by code when a level loads: the city, the buildings, the enemies, the weapons, the textures, the menus and most of the music. There are no imported 3D models. The fiction: Halcyon, a 1930s luxury monopoly, runs a Dark Art Deco city; you are a patient who walks out of its sanatorium; the enemies are the company's clockwork staff.

Numbers you will be asked for: about 260 code files, about 79,000 lines, 13 levels in three acts, one settings asset with roughly 850 tunable numbers, no third-party packages beyond Unity's own.

3. How the game is put together, in plain words

- **Loading a level.** A scene contains almost nothing. One script reads the settings asset and calls a level builder, which writes the whole level out of primitive boxes. Then it spawns the player, the HUD and the music. A redesign is a rewrite of the recipe, not a rebuild in an editor.
- **Levels are recipes.** A level file is about forty lines that read like a list: a ward, a cloakroom, a checkpoint, a city plan with named blocks (a diner, a speakeasy, a bathhouse, a tenement), the route, then a chute, a bridge and a hall. The kit underneath knows how to draw each of those. Acts: the city districts, the underground metro, the rooftops.

- **The movement engine.** A custom capsule is swept through the world every physics step. Air strafing adds speed when you turn into the direction you are steering (the Quake rule). Bunny hops, slides, a dash in any direction, slams, wall kicks, corner boosts and a parry all feed one speed. Every change of your speed is labelled with its cause. That label is what the score reads, and it is what lets the game notice a speed gain it did not plan for.
- **Combat.** Four classes chosen at a cloakroom: knuckles, the cane, the deck of cards, and the Set (blunderbuss, maul, harpoon gun, umbrella). Each weapon has a primary and an alternate cast; only the cane parries. Enemies are the company's staff: clockwork mannequins with a glass head as the weak spot.
- **Looks.** One art bible (cream, oxblood, brass, charcoal, black lacquer, stepped Deco forms) applied to everything. Every edge is drawn with a dark bead and every floor edge with a lit line, which is what lets boxes read as architecture.
- **Sound.** Recorded public-domain impact sounds layered by material. The music is composed at load: a noir jazz lead over hip-hop beds that harden as the fight does, plus twelve 1920s records as an optional gramophone layer.
- **Verification.** A compile check that builds the whole code base outside Unity in a minute, a build-time lint that checks every level is open and has several routes, a metrics log for combat tuning, and you, playing.

4. The design pillars (know the why for each)

1. **Skill-gated momentum.** Harder timing pays more. A perfectly timed hop multiplies speed; a lazy hop keeps it and gains nothing. Every weapon trick is capped so a chain of perfect hops always beats it. Why: otherwise the best play is to mash the strongest button, and the movement stops being the game.
2. **Hidden tech is rewarded, never patched.** Because every speed change is labelled, an unexplained gain is detected and scored. Physics exploits become features. Why: this is how speedrunning communities form, and you want them.
3. **No gates.** A level is one open line; enemies are fuel placed on the line, never waves that lock a door. The one exception is the boss arena at the end of Act I.
4. **A miss never kills.** Catch floors under every drop; hazards hurt and throw but never kill. The level is never the thing that stops you.
5. **The colour language.** Cyan means a cut target, gold means a perfect or flow, red means danger. The HUD shows state, never instructions.
6. **Dark Deco.** One bible for everything. Why: consistency is what makes primitives pass as a world.

5. The numbers (these are design numbers, so they are yours)

Thing	Value	Why it matters
Run speed	8 m/s	The baseline every multiplier is measured against
Jump	1.62 m high, 0.72 s in the air	Every ledge in the game is sized to it
Perfect hop window	plus or minus 50 ms	Multiplies speed by 1.18, tapering to 1.08 at high speed
Dash	18 m/s for 0.15 s, any direction	Costs one of three stamina pips
Parry	a 0.15 s beat inside a 0.35 s guard	Miss it and you are locked for 1.3 s
Blunderbuss	32 degree cone to 7 m, two shells	The blast jump is 10.5 m/s and is capped under a perfect hop
Highest paid move	the maul's rocket jump, 19.8 m/s up	Costs health and a pip
Levels	500 to 1,000 m long; gold times 75 to 220 s	A run is the five Act I levels in one go

6. The process (this is what you actually do, so be able to describe it)

1. You decide what the game needs, in plain words, with references (a Bloodthief level, a Deadlock corner boost, a Quarto de Hotel record).
2. The agent writes a design document from that, in the game's terms. You correct it. These documents are the contract: there is one for the movement rules, the level grammar, the districts, the company and its weapons, the art direction, the enemies, the UI.
3. Builders generate code against the documents, in parallel, each owning its own files. Every change is compiled by the external check before it counts.
4. Each builder reports what it built, what it could not verify, and what to play first.
5. You play. What feels wrong goes back as a sentence ("the fists do not even look like fists", "the weapons master's weapons feel clunky") and becomes a fix wave.

Have two of those feedback loops ready to tell as stories. Good ones: the music was silent because Unity imported the audio files before the importer that sets their format had been compiled, so every clip was stored in a form the game could not read; a log line gave it away and the fix was to rewrite the import settings and enforce them on every import. Or: weapon switching "did not work" because the one script that reads the switch key threw away any press made while the weapon was still animating in, so fast players never switched; the fix was to keep the press and apply it the moment the weapon arrives.

7. What is honestly not done (say this unprompted)

- Nobody but you has play-tested it. The latest wave (five district levels, the new weapons, a feel pass) compiles but has not been played yet.
- There are no automated gameplay tests. Verification is the compile check, the lint, the metrics log, and play.
- Performance has not been measured. Frame rates in the districts are unknown.
- The generated code has inconsistencies: duplicated helpers, long files, comments in several voices.
- Saves are a simple key-value store with no migration. The music is a stand-in for a real score. No controller support, accessibility or localisation.

8. Questions you will get

Did you write this? "I designed and directed it. The code is generated to my spec and review, and I can show you the design documents and the transcripts. I do not write C#."

Then what do you understand? "What every system does, why it exists, what the numbers are and why, what is broken, and how a change gets made and verified." Then show it: open the Level 1 recipe and read it line by line in plain words.

Can you change something right now? Yes. Open the settings asset in Unity's inspector, change the jump height or the perfect-hop window, press play, feel the difference. Those knobs exist precisely so a designer can tune without code.

Could you have done this without the AI? "No. What I bring is the design, the taste and the judgement; what the AI brings is the engineering. The skill I am building is direction and verification, which is what a lead does on any team."

How do you know the AI's code is right? "I do not take its word. It compiles outside Unity before it counts, the lint checks every level at build time, the metrics log shows whether a combat change helped, and I play everything. When it is wrong I find out the way any producer does: it does not feel right, and I say so."

What is the hardest problem in here? Pick the movement engine: making every speed change explainable so that the score, the hidden-tech reward and the fairness of the caps all rest on one fact. Or the level grammar: turning "a level" into a vocabulary of open spaces that a recipe can compose.

What would you do differently? Tests from the start; a real pipeline for the things that should not be generated (music, hero animations, which a friend is about to do in Blender); more play-testers earlier.

9. The demo, fifteen minutes

1. Main menu: the Programme, the thumbnails. Say the city behind the menu is generated too.

2. Practice range with F3 on. The readout names every move and its multiplier as you do it. Hop a chain, slide-hop, dash up, wall kick into a corner boost, parry a shot with the cane. Explain the skill gate while you do it.
3. Level 1: the ward, the cloakroom choice, the street door, then three routes through the district (the street, the alley, through the bathhouse).
4. Open `Assets/Scripts/Level1Builder.cs` and read it as a recipe. Open the settings asset and change one number live.
5. Run `Tools/compile.ps1` and say why it exists. Open the Docs/Design folder and show the design documents.

10. Before the meeting (two evenings)

1. Play every level you will show with F3 on until you can predict what the readout will say.
2. Have the agent read you `Level1Builder.cs` and `Level1District.cs` line by line, then explain them back without help. They are the one piece of code a non-coder can genuinely read, and reading them well is worth more than bluffing about the engine.
3. Read the design documents in Docs/Design in this order: skill-gated momentum, course grammar, district grammar, the House, the Deco direction. They are yours; be able to say what each decided and why.
4. Rehearse the demo once with a timer.

What not to do: do not say "I built this" without the qualifier; do not answer a code question with a guess, say what the part does and how you would get it changed; do not hide the weaknesses, list them before you are asked.